

Matthew Tran

Thread Synchronization Lab: Mutexes and Race Conditions

In this lab, I explored thread synchronization in C using POSIX threads (**pthread**). I learned how race conditions occur when multiple threads access shared data without coordination, and how to prevent them using mutex locks.

Tools Used

- C programming language
- POSIX Threads (pthread)
- gcc compiler
- Linux terminal

Skills Learned

- Creating and managing multiple threads using `pthread_create` and `pthread_join`
- Identifying and reproducing race conditions in concurrent programs
- Using mutex locks (`pthread_mutex_t`) to synchronize thread access to shared data
- Comparing execution times and results with and without synchronization
- Observing how loop size and thread timing affect race condition visibility
- Implementing thread-safe code to ensure data consistency in multithreaded applications
- Understanding the trade-offs between synchronization overhead and program correctness

Task 1) Single-Thread Counting

I created a single thread to perform 600,000,000 additions to a shared variable **sum**. The main thread waited for the counting thread to finish using **pthread_join**.

```

1  #include <stdio.h>
2  #include <pthread.h>
3
4  #define NUM_LOOPS 600000000
5  long long sum = 0;
6
7  void* countingThread(void* arg) {
8      for (int i = 0; i < NUM_LOOPS; i++) {
9          sum++;
10     }
11     return NULL;
12 }
13
14 int main() {
15     pthread_t tid;
16     pthread_create(&tid, NULL, countingThread, NULL);
17     pthread_join(tid, NULL);
18
19     printf("Sum = %lld\n", sum);
20     return 0;
21 }

```

```

matthew_tran@vm:~/.../Lab4$ make clean
rm -f count
matthew_tran@vm:~/.../Lab4$ make
rm -f count
gcc -Wall -g -std=c99 -Werror -pthread count.c -o count
matthew_tran@vm:~/.../Lab4$ gcc -Wall -g -std=c99 -Werror count.c -o count
/usr/bin/ld: /tmp/ccrsB6Da.o: in function `main':
/home/seed/Downloads/Lab4/count.c:19: undefined reference to `pthread_create'
/usr/bin/ld: /home/seed/Downloads/Lab4/count.c:20: undefined reference to `pthread_join'
matthew_tran@vm:~/.../Lab4$ gcc -Wall -g -std=c99 -Werror count.c -o count^C
matthew_tran@vm:~/.../Lab4$
matthew_tran@vm:~/.../Lab4$
matthew_tran@vm:~/.../Lab4$ gcc -Wall -g -std=c99 -Werror -pthread count.c -o count
matthew_tran@vm:~/.../Lab4$ time ./count
Sum = 600000000

real    0m0.932s
user    0m0.925s
sys     0m0.005s
matthew_tran@vm:~/.../Lab4$

```

The thread function **countingThread** runs a loop incrementing sum. With a single thread, I observed consistent and correct results. No race condition occurred because only one thread accessed the shared variable.

Task 2) Two Threads Without Synchronization

I created two threads, each incrementing sum 300,000,000 times, without any synchronization mechanism.

```

1 #include <stdio.h>
2 #include <pthread.h>
3
4 #define NUM_LOOPS 600000000
5 long long sum = 0;
6 pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
7
8 void* counting_function_up(void* offset) {
9     int increment = *(int*)offset;
10    for (int i = 0; i < NUM_LOOPS; i++) {
11        pthread_mutex_lock(&mutex);
12        sum += increment;
13        pthread_mutex_unlock(&mutex);
14    }
15    return NULL;
16 }
17
18 void* counting_function_down(void* offset) {
19     int decrement = *(int*)offset;
20    for (int i = 0; i < NUM_LOOPS; i++) {
21        pthread_mutex_lock(&mutex);
22        sum -= decrement;
23        pthread_mutex_unlock(&mutex);
24    }
25    return NULL;
26 }
27
28 int main(void) {
29    pthread_t thread1, thread2;
30    int offset = 1;
31
32    pthread_create(&thread1, NULL, counting_function_up, &offset);
33    pthread_create(&thread2, NULL, counting_function_down, &offset);
34
35    pthread_join(thread1, NULL);
36    pthread_join(thread2, NULL);
37
38    printf("Sum = %lld\n", sum);
39
40    pthread_mutex_destroy(&mutex);
41    return 0;
42 }

```

```

matthew_tran@vm:~/.../Lab4$ time ./count
Sum = 0

```

```

real    1m7.375s
user    1m38.107s
sys     0m35.337s

```

Since both threads modify the shared variable without any form of synchronization/teamwork, the situation is like a race. This task shows that when multiple threads access a shared resource without locks, the outcome can be unpredictable, depending on the timing of thread execution.

Task 3) Two Coinciding Threads (Separate Functions)

I launched two threads at nearly the same time, each running a separate counting function.

```
1 #include <stdio.h>
2 #include <pthread.h>
3
4 #define NUM_LOOPS 600000000
5 long long sum = 0;
6
7 void* counting_function_up(void* offset) {
8     int increment = *(int*)offset;
9     for (int i = 0; i < NUM_LOOPS; i++) {
10         sum += increment;
11     }
12     return NULL;
13 }
14
15 void* counting_function_down(void* offset) {
16     int decrement = *(int*)offset;
17     for (int i = 0; i < NUM_LOOPS; i++) {
18         sum -= decrement;
19     }
20     return NULL;
21 }
22
23 int main(void) {
24     pthread_t thread1, thread2;
25     int offset = 1;
26
27     // Create both threads
28     pthread_create(&thread1, NULL, counting_function_up, &offset);
29     pthread_create(&thread2, NULL, counting_function_down, &offset);
30
31     // Wait for both threads to complete
32     pthread_join(thread1, NULL);
33     pthread_join(thread2, NULL);
34
35     // Print the result
36     printf("Sum = %lld\n", sum);
37     return 0;
38 }
```

Show all screenshots and the screenshot for the time elapsed.

```
matthew_tran@vm:~/.../Lab4$ ./count
Sum = -75561522
matthew_tran@vm:~/.../Lab4$ ./count
Sum = 72158689
matthew_tran@vm:~/.../Lab4$ ./count
Sum = -4506423
matthew_tran@vm:~/.../Lab4$ ./count
Sum = -210458185
matthew_tran@vm:~/.../Lab4$ time ./count
Sum = -90859839
```

```
real    0m1.422s
user    0m2.777s
sys     0m0.012s
```

This task further highlights that starting threads at the same time without synchronization mechanisms does not prevent race conditions. The inconsistency shows the need for mutexes or other synchronization tools.

Task 4) Solving the Race Condition

I introduced a `pthread_mutex_t` lock. Each thread locks the mutex before updating `sum` and unlocks after.

```

1 #include <stdio.h>
2 #include <pthread.h>
3
4 #define NUM_LOOPS 600000000
5 long long sum = 0;
6 pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
7
8 void* counting_function_up(void* offset) {
9     int increment = *(int*)offset;
10    for (int i = 0; i < NUM_LOOPS; i++) {
11        pthread_mutex_lock(&mutex);
12        sum += increment;
13        pthread_mutex_unlock(&mutex);
14    }
15    return NULL;
16 }
17
18 void* counting_function_down(void* offset) {
19     int decrement = *(int*)offset;
20    for (int i = 0; i < NUM_LOOPS; i++) {
21        pthread_mutex_lock(&mutex);
22        sum -= decrement;
23        pthread_mutex_unlock(&mutex);
24    }
25    return NULL;
26 }
27
28 int main(void) {
29     pthread_t thread1, thread2;
30     int offset = 1;
31
32     pthread_create(&thread1, NULL, counting_function_up, &offset);
33     pthread_create(&thread2, NULL, counting_function_down, &offset);
34
35     pthread_join(thread1, NULL);
36     pthread_join(thread2, NULL);
37
38     printf("Sum = %lld\n", sum);
39
40     pthread_mutex_destroy(&mutex);
41     return 0;

```

```

matthew_tran@vm:~/.../Lab4$ time ./count
Sum = 0

```

```

real    1m1.856s
user    1m30.755s
sys     0m31.838s
matthew tran@vm:~/.../Lab4$

```

This task illustrates how mutexes can be used to prevent race conditions by controlling access to shared resources. Using `pthread_mutex` ensures that one thread completes its operation on `sum` before the other can access it, thus maintaining data integrity.

Task 5) Reduced Loop Count (20,000,000)

```
1 #include <stdio.h>
2 #include <pthread.h>
3
4 #define NUM_LOOPS 20000000
5 long long sum = 0;
6 pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
7
8 void* counting_function_up(void* offset) {
9     int increment = *(int*)offset;
10    for (int i = 0; i < NUM_LOOPS; i++) {
11        pthread_mutex_lock(&mutex);
12        sum += increment;
13        pthread_mutex_unlock(&mutex);
14    }
15    return NULL;
16 }
17
18 void* counting_function_down(void* offset) {
19     int decrement = *(int*)offset;
20    for (int i = 0; i < NUM_LOOPS; i++) {
21        pthread_mutex_lock(&mutex);
22        sum -= decrement;
23        pthread_mutex_unlock(&mutex);
24    }
25    return NULL;
26 }
27
28 int main(void) {
29     pthread_t thread1, thread2;
30     int offset = 1;
31
32     // Create both threads
33     pthread_create(&thread1, NULL, counting_function_up, &offset);
34     pthread_create(&thread2, NULL, counting_function_down, &offset);
35
36     // Wait for both threads to complete
37     pthread_join(thread1, NULL);
38     pthread_join(thread2, NULL);
39
40     // Print the result
41     printf("Sum = %lld\n", sum);
42
43     // Destroy the mutex
44     pthread_mutex_destroy(&mutex);
45
46     return 0;
47 }
```

This demonstrates that reducing the number of operations can lead to faster execution times, as expected. However, the importance of mutexes remains, which does not matter the number of iterations, to maintain accuracy in multithreaded environments.

Task 6) Race Condition with Same Offset

Purpose: The purpose of this task is to see what happens when the same offset is passed in two different threads.

```

1 #include <stdio.h>
2 #include <pthread.h>
3
4 #define NUM_LOOPS 20000000
5 long long sum = 0;
6
7 void* counting_function_up(void* offset) {
8     int increment = *(int*)offset;
9     for (int i = 0; i < NUM_LOOPS; i++) {
10         sum += increment;
11     }
12     return NULL;
13 }
14
15 void* counting_function_down(void* offset) {
16     int increment = *(int*)offset;
17     for (int i = 0; i < NUM_LOOPS; i++) {
18         sum -= increment;
19     }
20     return NULL;
21 }
22
23 int main(void) {
24     pthread_t thread1, thread2;
25     int offset = 1;
26
27     pthread_create(&thread1, NULL, counting_function_up, &offset);
28     pthread_create(&thread2, NULL, counting_function_down, &offset);
29
30     pthread_join(thread1, NULL);
31     pthread_join(thread2, NULL);
32
33     printf("Sum = %lld\n", sum);
34
35     return 0;
36 }

```

```

matthew_tran@vm:~/.../Lab4$ time ./count
Sum = 21279460

real    0m0.053s
user    0m0.097s
sys     0m0.004s

```

The task shows that even when threads are performing the same operation with the same offset, race conditions can lead to unreliable results. It shows the dangers of same time access to shared data without proper synchronization.

Task 7) Fixing with Mutex (Same Offset)

I added a mutex to Task 6's code.

```

1 #include <stdio.h>
2 #include <pthread.h>
3
4 #define NUM_LOOPS 20000000
5 long long sum = 0;
6 pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
7
8 void* counting_function_up(void* offset) {
9     int increment = *(int*)offset;
10    for (int i = 0; i < NUM_LOOPS; i++) {
11        pthread_mutex_lock(&mutex);
12        sum += increment;
13        pthread_mutex_unlock(&mutex);
14    }
15    return NULL;
16 }
17
18 void* counting_function_down(void* offset) {
19     int increment = *(int*)offset;
20    for (int i = 0; i < NUM_LOOPS; i++) {
21        pthread_mutex_lock(&mutex);
22        sum -= increment;
23        pthread_mutex_unlock(&mutex);
24    }
25    return NULL;
26 }
27
28 int main(void) {
29     pthread_t thread1, thread2;
30     int offset = 1;
31
32     pthread_create(&thread1, NULL, counting_function_up, &offset);
33     pthread_create(&thread2, NULL, counting_function_down, &offset);
34
35     pthread_join(thread1, NULL);
36     pthread_join(thread2, NULL);
37
38     printf("Sum = %lld\n", sum);
39
40     pthread_mutex_destroy(&mutex);
41
42     return 0;
43 }
44

```

```

matthew_tran@vm:~/.../Lab4$ time ./count
Sum = 40000000

real    0m2.158s
user    0m3.249s
sys     0m1.001s

```

This task shows the importance of mutexes when multiple threads access shared data. Mutexes help achieve expectable outcomes by preventing concurrent access to critical sections.

Task 8) Small Loop Count Without Mutex

I reduced loops to a very small number (1000) and ran without a mutex.


```

1 #include <stdio.h>
2 #include <pthread.h>
3
4 #define NUM_LOOPS 1000
5 long long sum = 0;
6
7 void* counting_function_up(void* offset) {
8     int increment = *(int*)offset;
9     for (int i = 0; i < NUM_LOOPS; i++) {
10         sum += increment;
11     }
12     return NULL;
13 }
14
15 void* counting_function_down(void* offset) {
16     int increment = *(int*)offset;
17     for (int i = 0; i < NUM_LOOPS; i++) {
18         sum -= increment;
19     }
20     return NULL;
21 }
22
23 int main(void) {
24     pthread_t thread1, thread2;
25     int offset = 1;
26
27     pthread_create(&thread1, NULL, counting_function_up, &offset);
28     pthread_create(&thread2, NULL, counting_function_down, &offset);
29
30     pthread_join(thread1, NULL);
31     pthread_join(thread2, NULL);
32
33     printf("Sum = %lld\n", sum);
34
35     return 0;
36 }

```

matthew_tran@vm:~/.../Lab4\$ time ./count

Sum = 2000

```

real    0m0.001s
user    0m0.001s
sys     0m0.000s

```

This task demonstrates that even with fewer iterations, mutexes are essential to ensure accuracy. However, it also shows that in low-iteration scenarios, the performance of mutexes may be less noticeable.